The class-contiguous parallel system — how it works, what it buys, and how to proceed

lie-rs / signature-rs — 2026-09-01. Every number is a fresh measurement of the current code; benchmarks: criterion baselines `anagram*/final`, fold probe (`PROF_GRID`).

The three layers

1. `ClassOrder` — An `Arc`-shared, basis-keyed plan: within each degree, words regrouped so every anagram class is one contiguous block — plus the relabeled scatter array, entry table, and degree table. Depends only on the **basis**: every series over the same basis shares one handle, so the O(basis) planning is paid once and reused across operands, kernel calls, folds, and batches of folds (`class_order()` is O(1) after the first call).

2. The kernel — Three monomorphized layouts behind one sweep: **direct** (public order, unchanged hot loop), **class→public** (class-layout scratch + overlapped per-job epilogue; the public-API path, auto-enabled when a degree slice exceeds L1), and **class→class** (internal mode: class-ordered operands, results written directly — no scratch, no permutation). Gating is shared: the degree-mask cache keys are layout-invariant.

3. The fold — The BCH DAG runs end-to-end in class order: operands and support lists converted once per fold, node buffers class-ordered, one dispatch per dependency level, and a single O(basis) epilogue (`accumulate_terms`) back to public order. The handle is created once per DAG and shared by every fold and every `LogSignature` clone.

Results are bit-identical across layouts and thread counts by construction: the layout relabels write **addresses** only — each result word is still accumulated by its own unit's entries in the same order. Verified at 1/2/4/8 threads on both kernel entry points and by the exact-arithmetic DAG oracle.

What it buys today (measured)

bench	before	after	×	4t ×	8t ×	regime
4×10 batch	156.6 ms	138.2	1.13	1.08	0.99	kernel: scatter leaves L1 (838 KB slice) ⟶ class blocks
5×8 batch	97.2 ms	81.6	1.19	1.00	0.89	kernel: same, larger slice (390 KB)
2×12 fold	4.29 ms	3.77	1.14	—	—	fold: no scratch/epilogue per node call
3×8 fold	3.47 µs	3.49	1.00	—	—	fold: dispatch-bound — nothing to win yet

Batch = 64 jobs, 2 threads (criterion medians; 5×8/3×10 vs `direct310`, 4×10 vs `postrace`). Fold = probe, 32-thread pool, steady state, vs the public kernel without the (deliberately removed) entry threshold. The pattern: wins scale with degree-slice size and **shrink with thread count** — at 8 threads the sweep is already near-ideal and the epilogue is pure added cost.

What constrains the next steps (measured, not guessed)

- A fold has only **12–31 anagram classes**, and **each DAG node is exactly one class** (a bracket only activates units with γ = γ_left + γright) — so per-level task lists are class-contiguous by construction; no sorting needed.
- Greedy 8-pack imbalance across a whole fold: **1.04–1.16×** — class-partitioned work packs balance almost perfectly.
- But the largest class holds **8–15% of the work**, capping pure class-affinity at 7–12× on 32 threads, and the DAG is a **funnel**: the first and last levels have 1–2 classes no matter the schedule.
- The small-fold cost is **dispatch churn**: 35% of a 3×8 fold's time is crossbeam-epoch/steal from per-level rayon dispatches of tiny task sets.

Decision menu

A. One-dispatch fold — built, gated — `FOLD_ENGINE=1`: all levels planned up front, one pack-slot walk with level counters — saves the per-level dispatches (35% of a small fold). **Blocked** by a scheduler starvation: waiting slots occupy rayon workers while queued lower-level packs starve (sleep/yield does not fix it — sleepers hold their workers). Fix = worker-park protocol or caller-participating work-sharing cursor. Correctness already proven (oracle + round-trip tests pass under it).

B. Cross-fold pipeline — biggest win — Batch of folds shares the team: fold N's narrow levels overlap fold N−1's wide ones; the class packs and the ordering handle are reused for free (same basis). Needs the worker-park fix from A first, or a persistent thread team with level barriers. This is the O(1)-amortized, full-machine version of the original goal.

C. Big-class splitting — 1.1–1.3× — The 8–15% classes cap affinity; splitting one class's node sweep across threads (unit bundles already do this) removes the ceiling but dilutes locality. Only worth it above 8 threads.

D. 4×12 evidence — overnight — The kernel win scales with slice size (4×10: 838 KB ⟶ 1.13×). 4×12's degree-12 slice is 11.2 MB ≈ 350× L1 — expect the largest single-thread kernel win — but its structure-constant build is 16 h. One `BENCH_BATCH_GRID=4x12` command runs it unattended.

Recommendation. A then B: the worker-park fix is small and unlocks both the small-shape folds (removes the last dispatch overhead) and the cross-fold pipeline (the only path to near-linear many-thread folds, since per-fold class parallelism is capped at 7–12× by class granularity). C only if profiling after B shows big-class stragglers. D is free evidence — start it before touching code.